SN Computer Science (2025) 6:63
https://doi.org/10.1007/s42979-024-03608-4

SURVEY ARTICLE

Perceived Impact of AI-Based Tooling on Software Development Code Quality

Boris Martinović[1] · **Robert Rozić**[2]

Received: 10 September 2024 / Accepted: 9 December 2024
© The Author(s), under exclusive licence to Springer Nature Singapore Pte Ltd. 2025

Abstract
Artificial intelligence (AI) is a powerful tool that can play a crucial role in software development. The integration of AI in that domain has allowed various AI-powered tools to emerge and have the potential to positively impact code quality. This article explores the perceived impact of AI tools on the quality of software development code. Through a survey conducted in various tech companies, the findings of this study aimed to provide a comprehensive understanding of the current state and potential future trends of artificial intelligence in software engineering. The overall results show that there is high satisfaction among developers using AI tools, with more than three-quarters of them stating that the adoption of these tools positively impacted their overall satisfaction and productivity in the software development sector. Although there are relatively mediocre values in terms of grades in development satisfaction with readability, maintainability, efficiency, and precision, there is an overall positive perceived outlook for AI tools. There is room for growth in both adoption and refinement, especially when talking about developer-centric tools.

Keywords Artificial intelligence · AI tools · Perceived AI impact · Software development · Code quality

Introduction

This paper is an extended version of the paper published at the International Conference on Digital Transformation in Education and Artificial Intelligence Application titled Impact of AI Tools on Software Development Code Quality [1].

The rise in popularity of artificial intelligence transformed the approach to the coding process of software development. The integration of AI in that domain has allowed various AI-powered tools to emerge and have the potential to positively impact code quality. Although very exciting and enticing, this technology is still in its early stages of adoption and exploration. The number of software developers using AI tools is increasing, as tools can offer various capabilities such as code generation, automated testing, bug detection, bug fixes, and more. Still, most common base use case for most of the AI-powered development tools is the automation of time-consuming coding tasks. To allow those features to be as useful and as accurate as possible, AI models are trained using large data sets from the code repository, which allowed them to understand common coding patterns.

Our research focuses on a crucial aspect of this technological integration: whether AI tools contribute to an improvement in code quality from the developer's perspective. It is very important to understand the impact and the importance of tools using artificial intelligence, not only because of recent emergence of said tools, but because of their potential impact on the development landscape. AI-based tools can provide different development experience to individual developers and development teams, optimizing workflow, leaving more time for complex tasks. On the other side of the coin, they could also potentially cause disruption in the software engineer job market, due to aforementioned optimizations. This is why it is important to see the perspective of software engineers working in the field on those important questions.

In order to research the impact of AI tools on software development quality, a survey was designed and distributed

✉ Boris Martinović
boris.martinovic@phd.sum.ba

Robert Rozić
robert.rozic@fpmoz.sum.ba

[1] Postgraduate Doctoral Programme, University of Mostar, Mostar, Bosnia and Herzegovina

[2] Faculty of Science and Education, University of Mostar, Mostar, Bosnia and Herzegovina

Published online: 06 January 2025

to software developers in various tech companies. Software developers are using artificial intelligence (AI) to minimize repetitive coding tasks that consume a lot of time. This enables them to focus more on the fundamental logic and architecture of the applications they are developing. This shift in focus could lead to improvements in the overall quality of the code. Previous research related to this topic has been conducted in terms of empirical studies [2] of AI-assisted tools. In the study, it was found that the latest versions of ChatGPT, GitHub Copilot, and Amazon CodeWhisperer at the time generate the correct code 65. 2%, 46. 3% and 31. 1% of the time, respectively.

The main argument of this research suggests that code quality increases when AI-based tools are used. This is based on the assumption that AI tools help refine the structure, logic, and readability of the code while also improving other aspects such as security and overall robustness. Following sections of this paper will discuss metrics and elements of code quality, such as readability, maintainability, efficiency, and accuracy.

The target audience for this research includes not only software developers, but also code testers and other professionals involved in the software development process. The findings of this study aim to provide these groups with valuable information on the current usage of AI tools in software development, the perceptions of these tools among practitioners, and potential future trends in this domain.

Similar Work

In recent years, several studies have explored the impact of AI tools on software development and code quality. However, this is still a relatively new area of research, with limited studies addressing the broader implications of AI-assisted tools on developers' perception of code quality beyond security concerns.

A systematic review of the literature by Negri-Ribalta et al. [3] examined how AI models affect the security of code generation. Their findings revealed that while AI can boost software development and improve coding proficiency, it does not offer a guarantee of security. In fact, some AI models were found to produce software with vulnerabilities, highlighting the importance of implementing security measures and processes such as code verification [3].

Another study by Almeida et al. [5] introduced AICodeReview, an IntelliJ IDEA [4] plugin that leverages GPT-3.5 for automated code assessment. Their research demonstrated the effectiveness of the tool in identifying logic discrepancies and syntactical errors. The preliminary evaluation showed that developers using AICodeReview were able to detect and refactor more code smells in less time compared to manual code reviews. This suggests that integrating AI-based techniques can streamline the code review process, potentially encouraging advances in overall software quality [5].

Pearce et al. [6] examined the security implications of using GitHub Copilot for code generation, finding that a significant portion of the generated code contained vulnerabilities. The study highlighted that while Copilot can improve productivity, it often produces insecure code, particularly in scenarios requiring careful handling of inputs or sensitive operations. The findings emphasize the need for improved security measures in AI coding tools and greater awareness of the limitations of developers [6].

Perry et al. [7] studied the impact of AI code assistants on code security, revealing that developers using GitHub Copilot were significantly more likely to produce vulnerable code (67%) compared to those without it (27%). Despite this, users reported higher confidence in their code's security with AI assistance. The study underscores the risks of excessive reliance on AI tools and highlights the need for better security training for developers [7].

Although existing studies predominantly focus on security vulnerabilities, our research broadens the scope by evaluating additional metrics such as readability, maintainability, efficiency, and accuracy. Moreover, we explore how developers perceive the quality of code written with the aid of AI tools. This comprehensive approach aims to fill a literature gap, offering insights into the overall developer experience and identifying opportunities for improvement in AI-based development tools.

Elements of Code Quality

This study, examines the impact of AI tools on software development, particularly focusing on code quality. Code quality is a multidimensional concept that is crucial to the success of software projects. In the research, four key components of code quality are examined: readability, maintainability, efficiency, and accuracy. These elements were selected based on their importance in software development and their potential to be influenced by AI tools. High code quality ensures that the software is not only functional, but also reliable, secure, and scalable. Poor code quality can lead to increased technical debt, making future modifications and debugging more challenging. The survey respondents, who included both users and non-users of AI tools, provided information on how these tools impact these aspects of code quality.

Readability

Code readability is a fundamental element of code quality. It is defined as the ease with which code can be logically read and understood by others. The readability of a

program is related to its maintainability and is a key factor in the overall quality of the software [8, 9]. Highly readable code tends to be less prone to errors, is more accessible for debugging and maintenance, and fosters better collaboration between team members. Enhancing readability can be achieved through proper indentation, formatting, and spacing, which contribute to a more consistent and visible code structure, thereby facilitating the debugging process [10]. Additionally, the strategic inclusion of comments, with concise explanations for each method, aids in the understanding and future modification of the code. Employing consistent naming conventions further improves readability by ensuring uniformity across the codebase. In addition, reducing the level of nesting within the code can significantly enhance its readability, making it easier for developers to follow the logic of the program [11].

Survey Question:

> To what extent do you believe AI tools contribute to improving code readability in your projects?

AI tools can improve the readability of the code by providing suggestions for clearer variable names, improving the code structure, and providing helpful comments. These tools also aid in enforcing coding standards, ensuring consistent formatting, and identifying redundant or confusing code segments. Furthermore, AI tools can assist in optimizing and simplifying complex logic and making the code easier to follow [12]. As a result, the use of AI tools can contribute to an overall improvement in code readability, positively impacting the quality of codebase and software development.

One of the studies underscores the capability of GitHub Copilot, an AI-powered tool, to produce code with readability comparable to that of human programmers. It points to AI's role in enhancing code readability, while stressing the need for programmers to review AI-generated code to maintain quality and maintainability [13].

GitHub's recent research on GitHub Copilot Chat reveals significant improvements in code readability when using AI assistance. The study found that 85% of developers felt more confident in their code quality, including its readability, when using GitHub Copilot and Copilot Chat. Developers reported that AI-assisted code was easier to read and understand, potentially leading to fewer errors and improved collaboration [14].

Maintainability

Maintainability is another crucial aspect of code quality. It can be defined as the ease with which a software system or component can be modified to be corrected, improved, or adapted to its environment [15]. Maintainability is essential for the long-term success and sustainability of software projects. It ensures that future modifications or updates can be made efficiently and without negatively impacting the overall functionality of the system. A key metric for evaluating maintainability is the ease with which changes can be made to the code while minimizing associated risks. Although an excessive line count can often indicate lower maintainability, this is not always the case, as well-organized and modular code can still be maintainable despite a larger number of lines. Best practices for improving maintainability include designing code as simple and straightforward as possible, making it easier to understand, modify, test, and operate. Regularly refactoring the code, properly documenting it to aid developers in understanding its structure, automating the build process, and leveraging automated testing are all effective strategies for enhancing maintainability [11].

Survey Question:

> Have you noticed changes in code maintainability since incorporating AI tools?

Clean, well-structured code generated by AI tools can enhance maintainability. AI tools have the potential to predict maintenance issues early and suggest code refactoring or improvements. In addition, AI can help enforce best practices, ensuring that the code remains modular, well documented, and aligned with maintainability standards.

Efficiency

Code efficiency refers to the ability of software to perform tasks quickly and effectively utilizing minimal system resources. It is directly tied to the performance and speed of the software, with quality often evaluated based on these factors. Improving efficiency involves removing unnecessary code and optimizing performance by reducing complexity and improving resource usage [16]. Efficient code not only increases speed, but also reduces memory consumption and system demands, making it easier to maintain and scale, while reducing costs and improving overall user experience. Efficient code is crucial as it simplifies maintenance, debugging, and scaling, while also lowering hardware requirements, power consumption, and associated costs. To achieve this, developers often employ techniques such as using appropriate data structures and algorithms, reusing code, and regularly profiling and optimizing the code [17].

Survey Question:

> Do you think AI tools have positively influenced the overall efficiency of your coding process?

Efficiency in coding involves optimizing the speed and utilization of resources, and AI tools are pivotal in enhancing this efficiency. AI-driven code analysis automates many tasks and introduces a new level of precision and effectiveness in software development [18]. AI tools optimize algorithms, identify performance bottlenecks, and suggest

improvements in code structure. They also recommend more efficient data structures and reduce redundant operations. Furthermore, AI-driven profiling and analysis assist developers in fine-tuning code, resulting in reduced memory usage and faster execution times.

Accuracy

Code accuracy is arguably the most important aspect of code quality. It refers to the correctness and precision of the code in performing its intended tasks [19]. Achieving high code accuracy is a complex task for AI, as it involves understanding the intricate logic and requirements of the software and ensuring that the code runs flawlessly under a variety of conditions [20]. Additionally, code accuracy requires thorough validation and testing to ensure that the code behaves as expected in all scenarios [21].

Survey Question

How confident are you in the accuracy and relevance of AI-generated code suggestions?

Accuracy in AI implementation is crucial to ensure that code performs as intended. Achieving high code accuracy involves understanding the intricate logic and requirements of software to ensure flawless execution under various conditions. AI tools can significantly enhance code accuracy by automating testing processes, detecting and fixing bugs, and identifying potential errors or vulnerabilities [22]. However, there is a concern that reliance on AI tools might lead to an increased dependency on automated solutions, potentially reducing the need for critical thinking and problem-solving skills among developers [23].

Traditional Methods vs. AI Influence

Traditionally, code quality has been ensured through manual methods such as peer code reviews, adherence to style guides, and writing tests. AI tools are now augmenting these practices, making error detection, test writing, and code review more efficient. This integration signifies a shift in traditional methods, where AI tools not only contribute to the quality of the code, but also improve the effectiveness of conventional quality assurance processes.

The advent of AI tools that generate tests and improve code quality has marked a significant change in traditional methods. By automating the generation of test cases and streamlining error detection, these tools reduce the time and effort required for manual testing. This increased efficiency encourages developers to write more comprehensive tests, which can lead to a higher overall standard of code quality. As a result, the integration of AI tools not only enhances the quality of individual codebases but also has the potential to improve code quality on a global scale, fostering better software practices across the industry.

Overview of Tools Discussed in the Survey

It is important to find a general overview of some of the popular and diverse AI-centric tools that are present on the market at the time of writing and the overall usage of those tools. This was important because of the potential outlook on AI depending on the use of those tools. All of them were chosen for their features and differences compared to each other. Although the majority of the mentioned AI tools could be all used for performing similar tasks, they can be divided into two categories: general purpose and developer-specific. As the name says, general-purpose tools can be used for performing a multitude of tasks and are used by a wider population. However, developer-specific tools are related to improving software development workflows and improving them. This classification makes it easier to compare both the usage of these tools by developers and to have better insight into developer habits.

General-Purpose AI Tools

General purpose AI tools have uses in different spheres of life and are practically things that all people with basic computer-using skills can get their hands on. For this research, four particular tools were classified as general-purpose AI tools. They are ChatGPT, Bing Copilot, Perplexity, and DALL-E. Because of their general-purpose nature, the aforementioned tools could be used to complete developer-specific tasks, such as code generation, bug fixing, explanation of code, generating diagrams, and other different uses.

ChatGPT is arguably the most popular AI-based tool in the current market. It is a chat-like interface where users input their prompts in a text box, and users get the response as a streamed text inside the chat. This approach is making it very easy to understand and familiar to a larger population, because the context of the "chat" is preserved. By doing this, it enables users to quickly iterate on their prompts and create a "conversational" flow of informational exchange and, ultimately, more precise answers. It offers multiple models, including GPT 3.5 and GPT4. GPT 3.5 version can be used free of charge, while the GPT 4 version requires a paid subscription [24].

Bing Copilot is a tool that is extremely similar in form to ChatGPT, but with a very distinct difference: it is built directly into the Microsoft Edge browser. Microsoft Edge browser is the default browser for Windows operating systems, which takes about 70% [25] of the whole operating system market, eclipsing the total numbers by MacOS, various GNU/Linux distributions and other desktop operating

systems. This provides a good benchmark for usage compared to ChatGPT itself, as it can be estimated that Bing usage will be much lower compared to ChatGPT, despite being more available directly to consumers [26].

Perplexity.ai is a new AI tool that aims to replace search engines by giving users direct answers to their questions without the need to go to their questions, among others. What differs Perplexity from ChatGPT is that it offers its sources to the data as well, making it easily verifiable as needed. Users can utilize Perplexity for multitudes of tasks, such as before mentioned answering basic questions or going deeper into topics with its Copilot search. Copilot allows follow-up questions with context, deeper answers, and more references [27].

DALL-E is the tool that is very specialized when compared to the previous ones. Where others aim to mainly give users a way to get the data in written form, DALL-E is using GPT models to generate images from text. DALL-E, allows users to create images and art from text prompts, allowing users to combine concepts, attributes, and styles within generated images. Additionally, it can edit existing images, create variations, and understand how objects evolve over time. DALL-E was in the survey to see how much developers are using image generation tools, since visual tools can also be an important part of the software development process [28].

Developer-Specific AI Tools

Tools used for developer-specific tasks are a lot more diverse in nature compared to general-purpose tools mentioned in the section above. They offer different amounts of integration into developer workflows and can be used at various stages of development. Some of them are integrated development environments themselves, while others integrate into development environments or code editors, and some are used to generate the project initially. Tools that are covered are GitHub Copilot, Google IDX, Codeium, Wasp Mage, and Cursor.

GitHub Copilot is the the most popular AI-based tool that is specific to development workflows. It is not a standalone product, but has to be used within integrated development environment or a code editor. Currently supported editors are Visual Studio Code, Visual Studio, NeoVIM and in various Jetbrains IDEs. GitHub Copilot [29] is available with a subscription for both private users and companies. It works with different programming languages, claiming that it supports all languages that appear in public repositories. For some, like JavaScript, GitHub Copilot offers better support, since it is prominently used in public repositories. It offers support for various operations, such as explanation of code features, completion of code snippets, creation of tests with the code [30].

Google IDX unlike GitHub Copilot is a web-based integrated development environment that runs in a user's web browser environment and executes in the cloud. It supports projects in React, Next.js, Angular, Flutter, Vue, Svelte, Go, Python, and more [31]. It allows for both web development and multi-platform development and allows users to utilize built-in AI tools that aim to enhance both speed of writing and quality of code. At the time of writing, it is in the public beta phase [32].

Codeium is an AI-powered toolkit that supports AI code completion, search capabilities, and an AI chat function for developers. It shares some of its features with GitHub Copilot, namely auto-complete and chat features. The main difference and the reason why it is included is the fact that it has a free tier, which can make it appealing to developers who want to try out AI tools for the first time, without spending money [33].

Wasp Mage is vastly different tool compared to the other mentioned in this study. Wasp (Web Application Specification) is a tool that aims to simplify development of full-stack applications by combining the main.wasp configuration file with the React and Node.js files code in order to output a full-stack application consisting of React/Node.js/PostgresDB. The main.wasp file contains declaration for important parts of the application, such as full-stack authentication, database schema, asynchronous jobs, full-stack type safety. Wasp Mage is primarily used to kick start the development process by creating a full-stack app based on the initial prompt explaining the purpose of the application. The resulting output of the whole process is a Wasp application [34, 35].

Cursor is code editor that aims to integrate AI into various software development processes. It is a standalone development environment that allows users to ask for information about the codebase, automatically completing code snippets, chat, automatic code debugging, and more. Since it is a development environment for desktop use, it is useful to compare it directly with a similar cloud product, such as IDX [36].

Experiment

The main way to access the data for this experiment is through a survey. Survey itself is divided into three parts: demographic information, section for participants not using AI tools, and a different one for using AI tools.

For non-users of AI tools, it was important to identify reasons for not adopting these technologies and what could encourage their future use. Understanding these perspectives helps contextualize the adoption barriers and potential areas for improvement in the development of AI tools.

The demographic information section will provide information about age group, country of residence, current employment status and primary business sector of the participant's business. This information is gathered to form potential relationships between responses and to determine whether there are any correlations between the participant's demographic information and the use and satisfaction with AI tools. At the end of the section, there are two questions that provide more information about the participant's outlook on AI. One asks about the fear of AI replacing the participant in terms of work or employment, and the other inquires about the frequency of usage of AI in participant's software development workflows.

The section for participants not using AI tools will be filled by participants who answered "Never" on the question that asks "How frequently, on average, do you use AI-powered tools in your software development workflows?". This section is equally important for this study, as it will create the environment where participants will share their reasons for not using AI and also things that need to be improved in AI tools in order for participants to become users of AI-powered tools.

The section for participants who use AI is the main section of this survey. It is designed to find out the ways that AI-tools affect participant's software development workflow. The questions are related to the usage of particular AI tools mentioned in "Overview of Tools Discussed in the Survey" section of this paper and how tools that participants use affect their software development workflow. Before going to tools that affect code quality, the goal is to find out where participants use AI tools and how they pay for it. Participants can specify whether they use AI tools for business-related projects, private projects, or both. They can also state how they finance those tools: through the company they work out, paying for licenses out of their own pocket, or whether they only use free tools. Besides effects on the code quality, which was explained in "Elements of Code Quality" section of this paper, there are questions regarding AI tools influencing overall process and participant's satisfaction with them. It is also important to gauge the impressions of participants about the overall impact of AI for them as developers. For that, there are questions that pertain to recommending the AI tools that participants use themselves to others and how did AI tools impact participant's software development workflows.

The means of distributing this survey was conducted through multiple channels. The snowball sampling method was used as the optimal method for this survey. It was chosen because of specific participants that were needed (software engineers and developers) and to also maximize the reach since the survey was designed to last for 7 days. The survey was posted and shared on LinkedIn [37], X (formerly Twitter) [38] and personally shared with people and companies. The main target audience were people who write software, whether it is professionally or as a hobby, who have reached the adult age of 18 years. The survey itself did cover the potential for users who both use and don't use AI tools in their software development, so any distinction based on that was not needed. Also, the potential distribution of users and non-users of AI tools can be a potential interesting point for the results.

Results

In the survey, which lasted from February 20th to 28th 2024, responses were collected from 84 participants. Their responses revealed several key insights into the use of AI tools in software development and their impact on perceived code quality. Following subsections present the data and results of the survey.

Demographics

Figure 1 represents the age groups of the participants, with half of them being in range 26–35 years old. The participants in this survey were various professionals from different countries and age groups, mainly from Bosnia and Herzegovina and Croatia, as it is represented on Fig. 2. It is therefore important to emphasize the mostly regional nature of the participants, since 81% of all respondents come from those two countries. The majority of the respondents were between the ages of 25 and 45, representing a diverse range of experience levels in the field of software development.

Business Sector and Work Positions

As it is visible from Fig. 3, when examining the primary business sectors of the respondents, a substantial majority (75%) work in the IT sector, followed by 8.3% in the finance

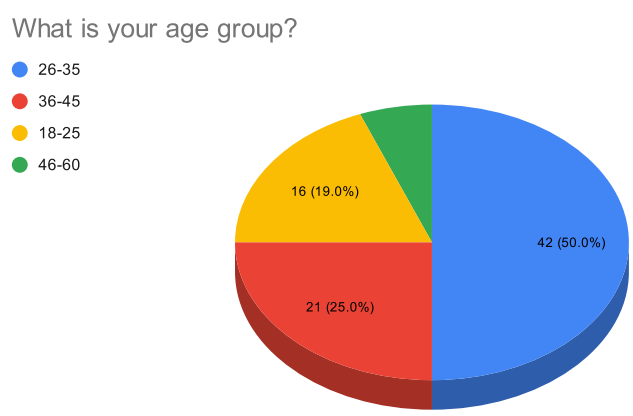

Fig. 1 Age group of respondents

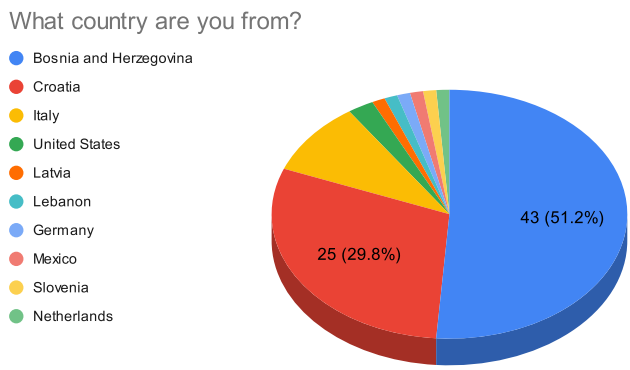

Fig. 2 Respondents country of residence

sector and 7.1% in education. These results underscore the diverse applications of AI tools in multiple sectors.

Figure 4 shows that participants have reported a wide range of job positions within various industries. Most of the respondents identified themselves as senior engineers or junior to mid-level engineers, reflecting a varied mix of expertise and roles within the industry.

Frequency of Usage of AI Tools

When asked about how often participants use AI tools, only 7.2% (6) study participants responded that they do not use them at all, and more than 59% (49) of them use AI tools on a daily basis, as the Fig. 5 shows.

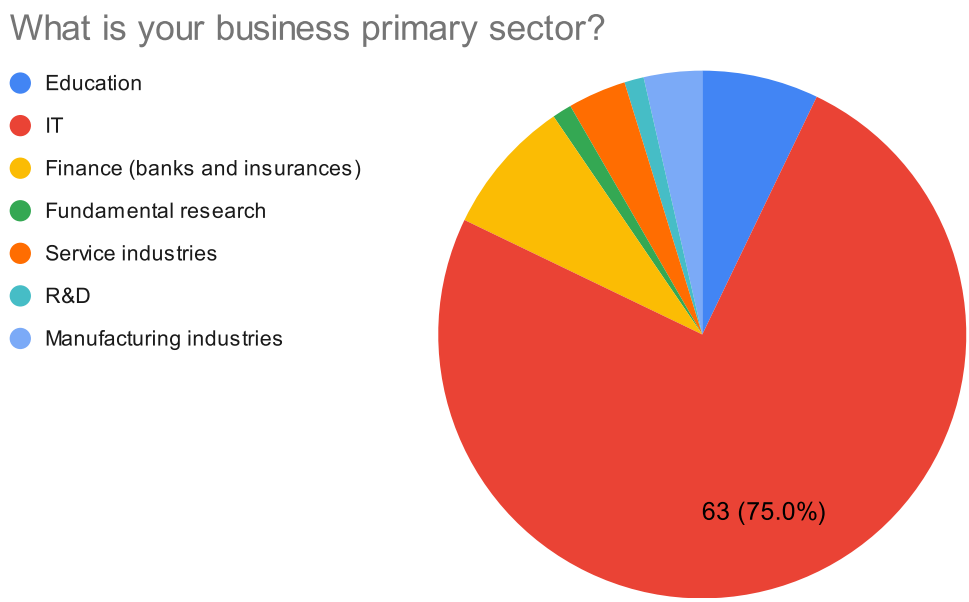

Fig. 3 Respondents business sector

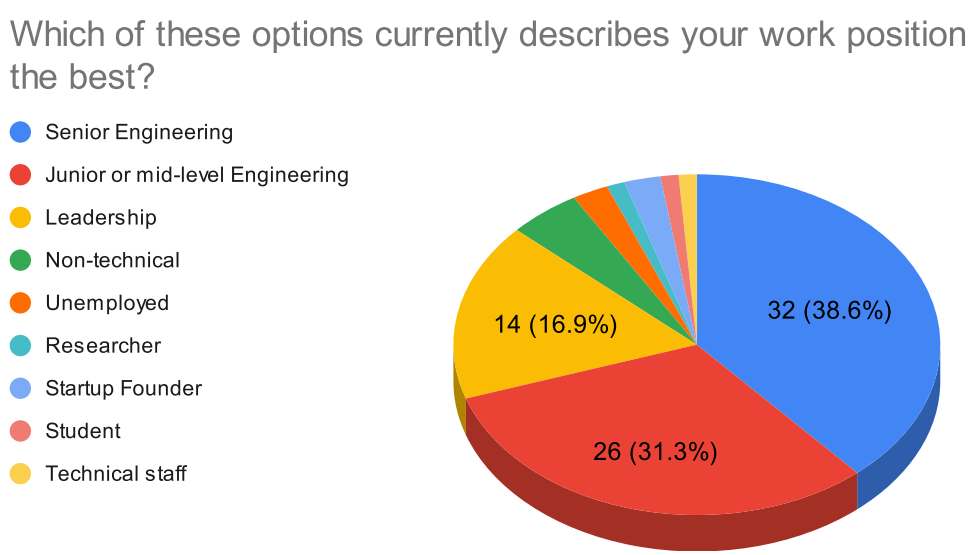

Fig. 4 Respondents work positions

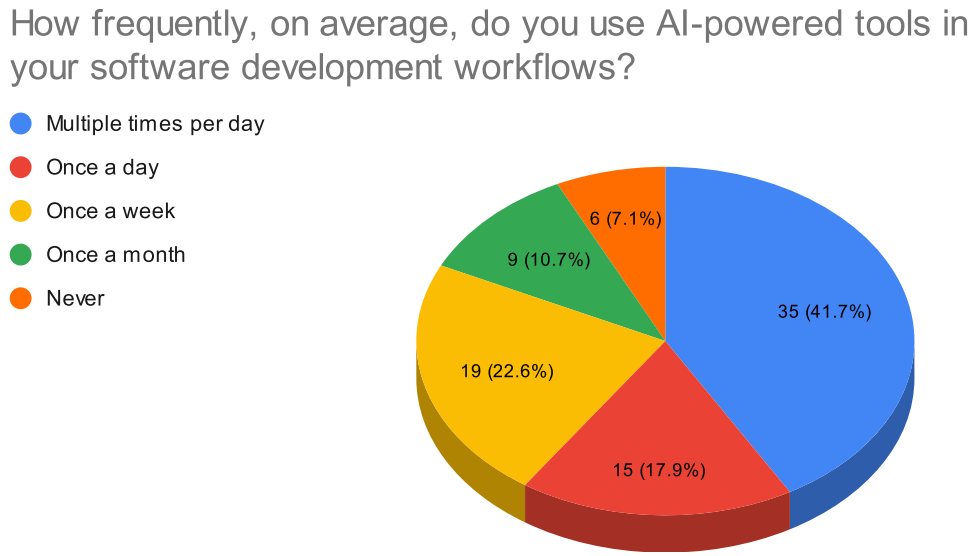

Fig. 5 AI tools frequency usage

The frequency of use of AI tools indicates strong integration into daily workflows, and many participants use the AI tools multiple times per day.

Most of the survey participants expressed no fear that AI would replace their jobs in the near future, despite their daily use of these technologies. Figure 6 shows that they see them as tools to enhance and speed up the development process and reduce the time wasted on time-consuming tasks with a greater focus on the architecture of the application and fundamental logic.

Popularity of AI Tools

Figure 7 shows that among the survey respondents, ChatGPT was shown to be the most used general-purpose AI tool, illustrating its widespread adoption in the software development sector.

Similarly, Fig. 8 demonstrates that GitHub Copilot stands out as the leading developer-specific AI tool, showing a significant trend in the industry.

Regarding payment for AI tools, the survey revealed a nearly balanced distribution, with 45% of users using paid versions, while a slight majority of 55% prefer the available free tools.

There is no clear dominant pattern in the use of AI tools for personal or business projects. 9% (7) use them only for personal purposes, 17% (13) for company projects, and the other uses them for a combination of both types of projects.

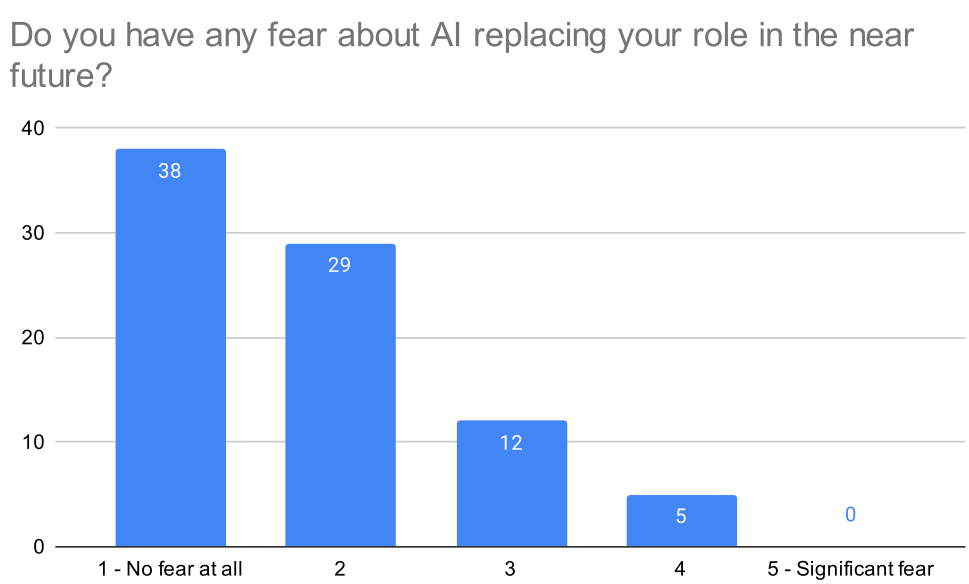

Fig. 6 Fear of being replaced by AI

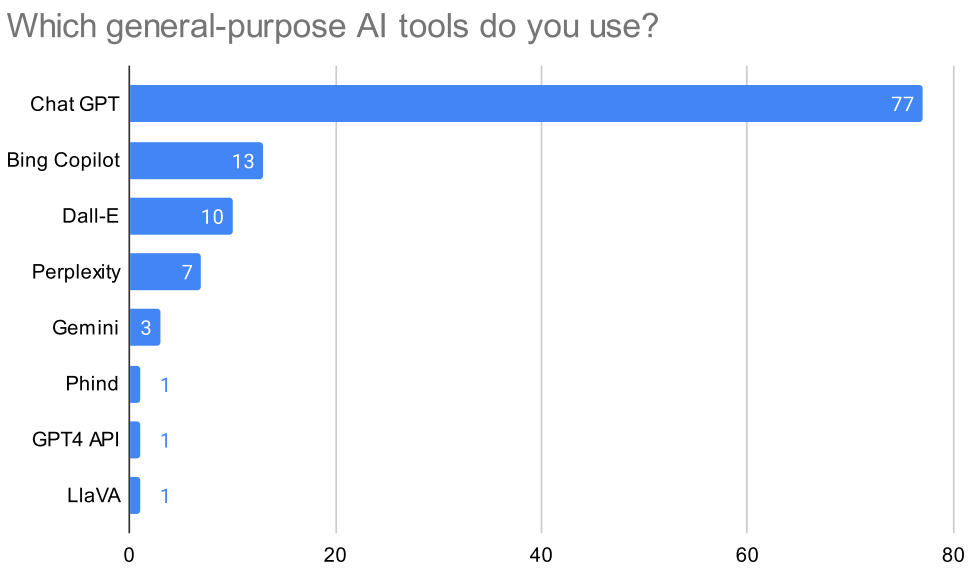

Fig. 7 General purpose AI tools usage

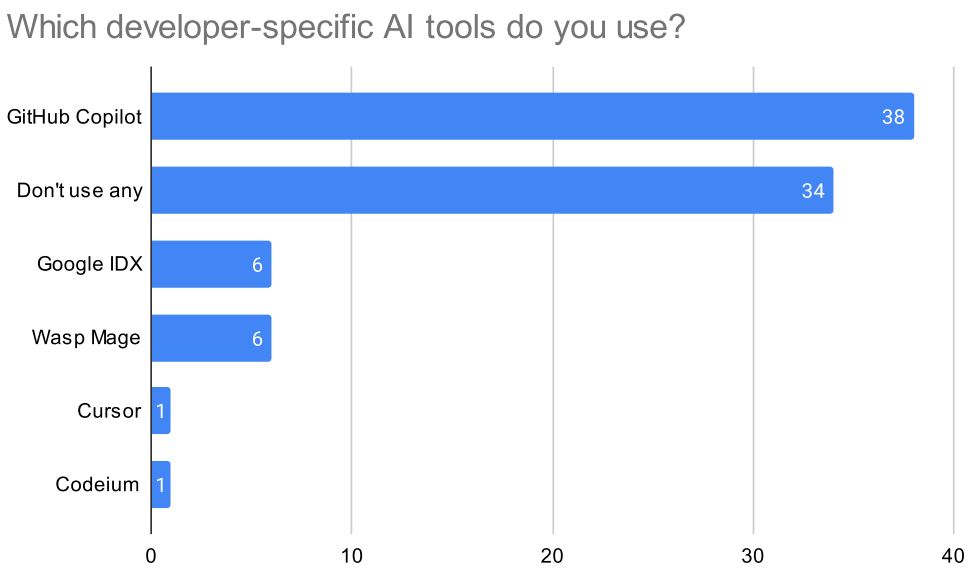

Fig. 8 General purpose AI tools usage

Self-Perceived Impact on Code Quality

Following graphs show self-perceived impact of AI tools on the code quality elements that have been used as a metric in this study: readability is shown in Fig. 9, maintainability in Fig. 10, efficiency in Fig. 11 and accuracy in Fig. 12.

Impact on Overall Satisfaction

Figure 13 represents the impact of AI tools on overall satisfaction and productivity in software development. Out of 78 participants using AI tools, more than 78% (61) have reported a positive influence on overall satisfaction and productivity in software development. The general sentiment of the respondents is that they are willing to recommend AI tools to their peers to improve code quality. This suggests that AI tools are not only enhancing individual productivity, but are also perceived to have a broader impact on the quality and maintainability of the codebase, contributing to more efficient and robust software development processes across teams and projects.

Perspective of Non-users of AI Tools

Our survey yielded a small group of 7 respondents who do not use AI tools in their software development processes. The main reasons include concerns about trust, cost, and lack of information on the benefits of AI tools. To encourage adoption among this group of respondents, improvements in

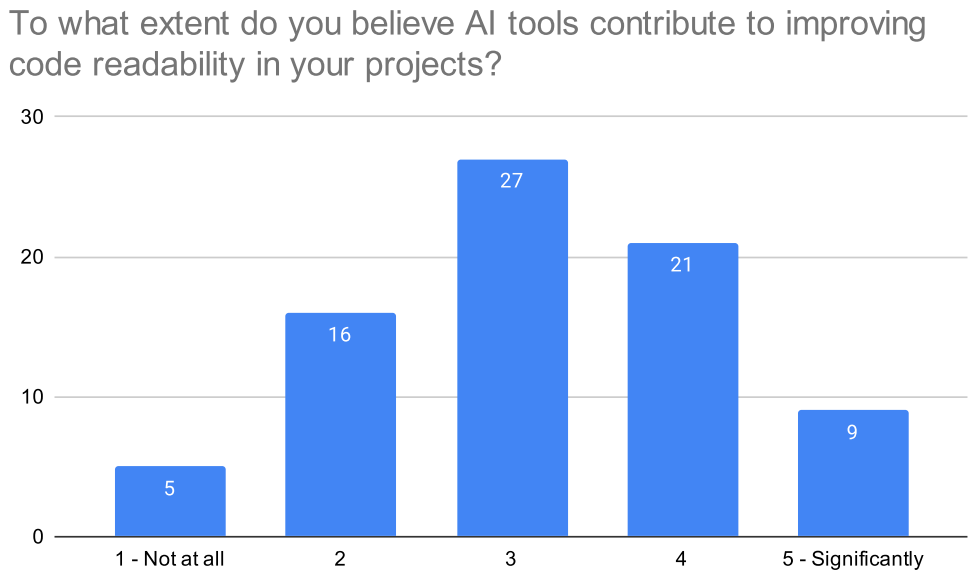

Fig. 9 Self-perceived impact on code readability

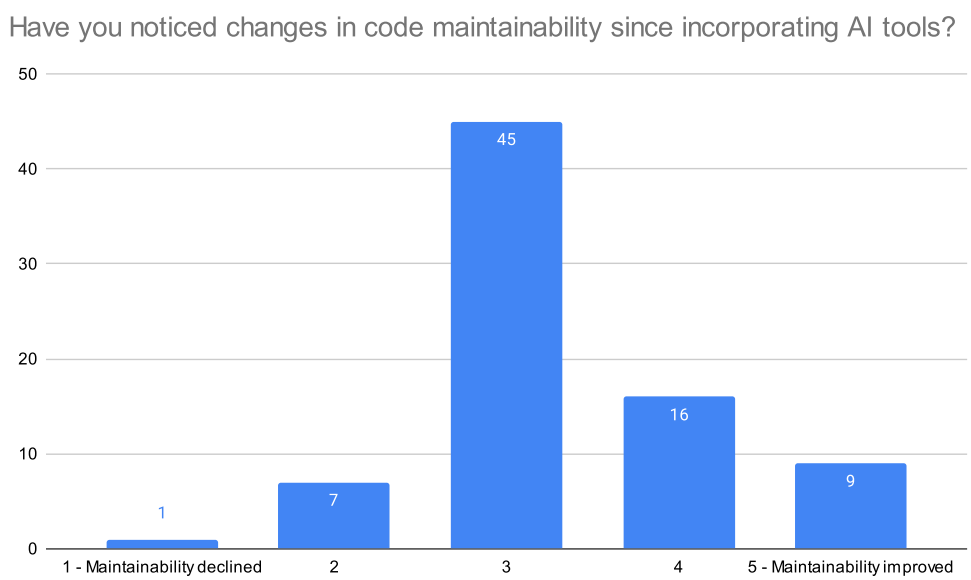

Fig. 10 Self-perceived impact on code maintainability

code quality, user experience, and affordability of AI tools were identified as key factors.

Conclusion

The rapid rise of AI tools also brought about some new and interesting ways of working for developers. At the moment, many software developers use some form of AI in their workflow, especially OpenAI's ChatGPT. The general findings among the users of AI tools in software development are very interesting. Developer-specific tools, such as GitHub Copliot, are less widely used compared to ChatGPT, meaning that there is much room for growth in terms of adoption and use, despite being more adjusted for specific development needs. The overall results show that there is high satisfaction among developers using AI tools, with more than three-quarters of them stating that the adoption of AI tools positively impacted their overall satisfaction and productivity in software development.

On the other hand, there is a different pattern in terms of satisfaction with individual elements of code quality. Most of the answers show that the AI tools are still not providing a significant number of users with a sufficiently high level of satisfaction. The average scores on the scale of 1 to 5 in terms of satisfaction are 3.17 for readability, 3.32 for maintainability, 3.85 for efficiency, and 2.92 for accuracy. Those grades are not necessarily bad considering the maturity of technology, but show a definite need to grow in all areas. Of all of them, efficiency is the most positive, mainly

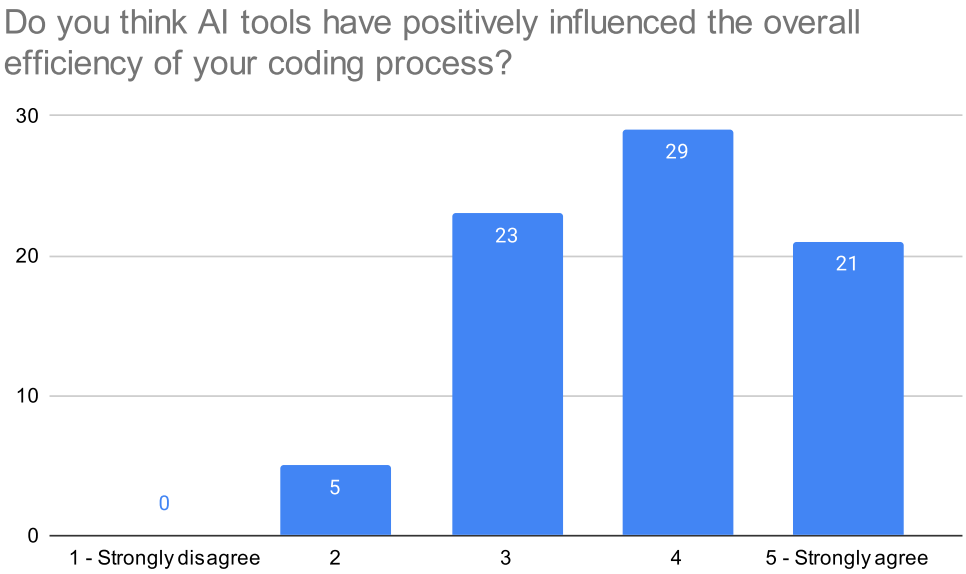

Fig. 11 Self-perceived impact on code efficiency

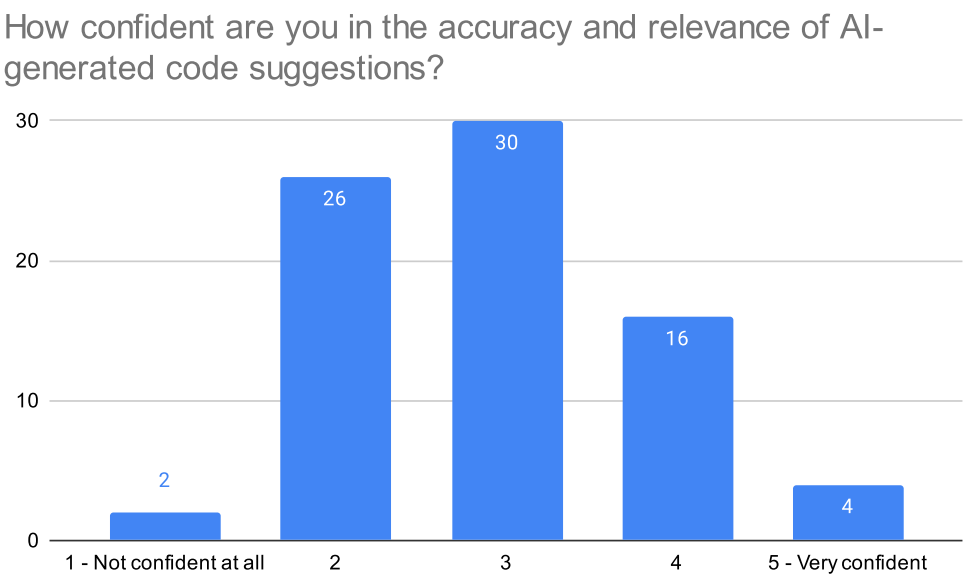

Fig. 12 Self-perceived impact on code accuracy

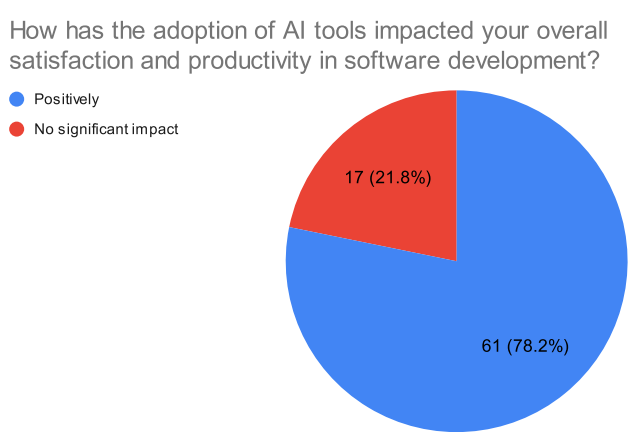

Fig. 13 Overall satisfaction

because AI tools speed up development. When talking about the accuracy score, it is important to note that it is consistent with previous research by Pearce et al. [6] and Perry et al. [7] showing vulnerabilities in code produced by using AI-based tools. Similarly, Pearce et al. [6] highlight that there is an improvement in productivity when using AI-based tools, which is consistent with the efficiency score.

Because of those results, it is important to outline the next steps to improve the AI agents themselves, so that they can improve the findings of this research. Firstly, before actually suggesting the code, AI tools can incorporate a verification step which will build and/or run the code which will be suggested. In that way, the accuracy of the code should improve, considering it will be able to execute. Code readability can be improved by AI tools that have access to the context of

the codebase by analyzing the ways and patterns in which variables in the file or projects are named and applying that rule set onto their output. By doing this, AI tools can be as true to the spirit of naming convention as possible. Writing snippets as functions that can be reused will help maintain the code maintainability. By ensuring that the functions use as much parametrization as possible, AI tools can be as maintainable as possible from the start.

These results show how the perceived value of all elements is not directly related to satisfaction and the willingness to recommend AI tools to others. Despite having relatively mediocre values in terms of grades in development satisfaction with readability, maintainability, efficiency, and precision, there is a positive result in terms of overall outlook on AI tools. With current data, it is visible that developers feel like AI tools are, in fact, useful to them and that they enhance the developer workflow.

When it comes to software developers who do not use AI tools in their workflow, they cite reasons such as not being able to afford paid tools and not trusting the tools as the main reasons. From both previous research and the one presented in this article, it is visible that those claims are not without merit. In the end, users decide for themselves which degree of code quality they can accept from the AI tools to be useful. Building on the findings, it's clear that while AI tools are gaining traction among software developers, there remains room for growth in both adoption and refinement, especially when talking about developer-centric tools.

Concerns from non-adopters among with areas for improvement: better-tested code, more accuracy, affordability, and user experience. Addressing these can broaden AI tool adoption. As developers see AI benefits like productivity and satisfaction, the user gap may close. Focusing on these could integrate AI tools more into software development, boosting code quality and efficiency.

Future Work

This paper is a stepping stone into further research. There are different ways of deepening the understanding of the findings made in this paper. Considering the fact that 81% of all participants come from only two countries, it is important to acknowledge the geographic limitations of the results of this research. This survey's population was mostly consisting of developers from Croatia and Bosnia and Herzegovina among developers from European Union countries and the USA. The population of research would have to be broadened to also affect other parts of the world, such as Asia, Africa, and Latin America. On the other hand, there is also a need to find deeper connections between high overall satisfaction with AI tools, among with relatively mediocre ratings of the aspects of code quality.

Besides obvious extensions to the current research, there are multitudes of subjects that surround the relations between software developers and artificial intelligence which could be explored in the future research. One of them is finding out what new features developers want in order to gain more productivity and better development experience. What is also important to find out is the sentiment regarding job security, whether software developers are afraid for their jobs and directly avoid using or suggesting AI tools because of that. Finally, there is also an aspect of comparing freelancer and full time employee productivity differences and whether single developers are able to directly earn more with the help of artificial intelligence tools.

In order to gain fuller picture, there are perspectives yet untouched by this research which are important, and that is the perspective from the employer side as well. One closely related research to this paper is researching the impact of AI tools used by software developers on employer satisfaction. Discovering whether AI tools improve the perception of productivity and satisfaction for software engineers will be crucial in order to actually decide whether employers should invest into AI tools for their software engineers at all. Concerns and questions can arise beyond the productivity and cost. Also, it would be interesting to find out whether there were layoffs of developers directly related to the use of AI-based tools at work.

In the modern world, there is heavy emphasis on security as well, especially in heavily regulated markets, such as the European Union. Connected to the security is also the question of perception of the code security, whether there are potential fears of private company code leaking outside the company. Companies invest heavily into their products and services, so it is natural they don't want their non-open-source code to go out. Beyond the code itself, some private keys or similar secrets are a potential leak threat, so it is very important that tools that developers use support that.

Funding Not applicable - no funding was received.

Availability of Data and Materials The anonymized survey results can be requested by sending an email to one of the authors.

Declarations

Conflict of interest Not applicable.

Research Involving Human and/or Animals Survey participants were informed before filling in the survey that the aggregated results of this survey will be published in the form of a paper. No personal data was published as part of the paper.

Informed Consent Survey participants were informed before filling in the survey that the aggregated results of this survey will be published in the form of a paper.

References

1. Martinović B, Rozić R. Impact of AI tools on software development code quality. In: International conference on digital transformation in education and artificial intelligence application. Springer; 2024. p. 241–256.
2. Yetiştiren B, Özsoy I, Ayerdem M, Eray T. Evaluating the code quality of ai-assisted code generation tools: an empirical study on github copilot, amazon codewhisperer, and chatgpt. 2023.
3. Negri-Ribalta C, Geraud-Stewart R, Sergeeva A, Lenzini G. A systematic literature review on the impact of AI models on the security of code generation. Front Big Data. 2024;7:1386720.
4. IntelliJ IDEA - the Leading Java and Kotlin IDE — jetbrains.com. https://www.jetbrains.com/idea/. Accessed 3 Dec 2024.
5. Almeida Y, Albuquerque D, DantasFilho E, Muniz F, de Farias Santos K, Perkusich M, Almeida H, Perkusich A. AICodeReview advancing code quality with AI-enhanced reviews. SoftwareX. 2024;26: 101677.
6. Pearce H, Ahmad B, Tan B, Dolan-Gavitt B, Karri R. Asleep at the keyboard? assessing the security of github copilot's code contributions. In: 2022 IEEE symposium on security and privacy (SP). IEEE; 2022. p. 754–768.
7. Perry N, Srivastava M, Kumar D, Boneh D. Do users write more insecure code with AI assistants? In: Proceedings of the 2023 ACM SIGSAC conference on computer and communications security. 2023. p. 2785–2799.
8. Buse RPL, Weimer WR. Learning a metric for code readability. IEEE Trans Softw Eng. 2010;36(4):546–58.
9. Tashtoush Y, Odat Z, Alsmadi IM, Yatim M. Impact of programming features on code readability. 2013.
10. What is Code Readability? - A Key Pillar of Software Development - Galaxy Weblinks — galaxyweblinks.com. https://www.galaxyweblinks.com/blog/what-is-code-readability. Accessed 3 Dec 2024.
11. 13 Code Quality Metrics That You Must Track | Opsera — opsera.io. https://www.opsera.io/blog/13-code-quality-metrics-that-you-must-track. Accessed 27 Aug 2024.
12. Improve Code Readability with AI | Restackio — restack.io. https://www.restack.io/p/ai-improve-answer-code-readability-cat-ai. Accessed 3 Dec 2024.
13. Al Madi N. How readable is model-generated code? examining readability and visual inspection of github copilot. In: Proceedings of the 37th IEEE/ACM international conference on automated software engineering. 2022. p. 1–5.
14. Rodriguez Mario. Research: Quantifying GitHub Copilot's impact on code quality — github.blog. https://github.blog/news-insights/research/research-quantifying-github-copilots-impact-on-code-quality/. Accessed 3 Dec 2024.
15. Ardito L, Coppola R, Barbato L, Verga D. A tool-based perspective on software code maintainability metrics: a systematic literature review. Sci Program. 2020;1–26:2020.
16. Dhaduk Hiren. Code Quality: Its Importance in Custom Software Development — simform.com. https://www.simform.com/blog/code-quality/. Accessed 28 Feb 2024.
17. Code Efficiency — devx.com. https://www.devx.com/terms/code-efficiency/. Accessed 27 Aug 2024.
18. Bielińska A. Code Analysis with AI. How to Do IT Effectively? - Edge1S — edge1s.com. https://edge1s.com/blog/code-analysis-with-ai/. Accessed 3 Dec 2024.
19. Bhatt T. What is Code Quality & What's Its Importance in Development? — intelivita.com. https://www.intelivita.com/blog/importance-of-code-quality/. Accessed 3 Dec 2024.
20. da Cruz WS. How AI is revolutionizing custom software requirements analysis - Ubistart — ubistart.com. https://ubistart.com/en/blog/how-ai-is-revolutionizing-custom-software-requirements-analysis/. Accessed 3 Dec 2024.
21. Vallejo L. Validation and Verification of Artificial Intelligence — sqs.es. https://www.sqs.es/validation-and-verification-of-artificial-intelligence/?lang=en. Accessed 3 Dec 2024.
22. Ensuring Accuracy: How to Check Code for AI Implementation. — texta.ai. https://texta.ai/blog/ai-content/ensuring-accuracy-how-to-check-code-for-ai-implementation. Accessed 3 Dec 2024.
23. Binmile Technologies. Pros & Cons of AI Assisted Coding | Binmile Technologies — binmile.com. https://binmile.com/blog/pros-and-cons-of-ai-assisted-coding/. Accessed 3 Dec 2024.
24. Introducing ChatGPT — openai.com. https://openai.com/blog/chatgpt. Accessed 28 Feb 2024.
25. Desktop Operating System Market Share Worldwide | Statcounter Global Stats — gs.statcounter.com. https://gs.statcounter.com/os-market-share/desktop/worldwide. Accessed 28 Feb 2024.
26. Your Everyday AI Companion | Microsoft Bing — microsoft.com. https://www.microsoft.com/en-us/bing. Accessed 28 Feb 2024.
27. Getting Started with Perplexity — blog.perplexity.ai. https://blog.perplexity.ai/getting-started. Accessed 28 Feb 2024.
28. DALL.E: Creating images from text — openai.com. https://openai.com/research/dall-e. Accessed 28 Feb 2024.
29. GitHub Copilot . Your AI pair programmer — github.com. https://github.com/features/copilot. Accessed 28 Feb 2024.
30. Rizel S. 8 things you didn't know you could do with GitHub Copilot — github.blog. https://github.blog/2022-09-14-8-things-you-didnt-know-you-could-do-with-github-copilot/. Accessed 24 Feb 2024.
31. Project IDX — idx.dev. https://idx.dev/. Accessed 28 Feb 2024.
32. Introducing Project IDX, An Experiment to Improve Full-stack, Multiplatform App Development — developers.googleblog.com. https://developers.googleblog.com/2023/08/introducing-project-idx-experiment-to-improve-full-stack-multiplatform-app-development.html?m=1. Accessed 28 Feb 2024.
33. Codeium; Free AI Code Completion & Chat — codeium.com. https://codeium.com/. Accessed 28 Feb 2024.
34. Introduction | Wasp — wasp-lang.dev. https://wasp-lang.dev/docs. Accessed 28 Feb 2024.
35. MAGE GPT Web App GeneratorMageGPT — usemage.ai. https://usemage.ai/. Accessed 28 Feb 2024.
36. Cursor - The AI-first Code Editor — cursor.sh. https://cursor.sh/. Accessed 28 Feb 2024.
37. LinkedIn: — linkedin.com. linkedin.com. Accessed 20 Nov 2024.
38. x.com — x.com. https://x.com/. Accessed 20 Nov 2024.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Reproduced with permission of copyright owner. Further reproduction prohibited without permission.